Control 1 CC4303
Parte 1 — Proxy HTTP

Integrante: Julián Proboste

Introducción

Un proxy es un dispositivo intermedio entre un cliente y un servidor que realiza las consultas a nombre del cliente y luego se las reenvía. Para ello debe ser capaz de recibir consultas como lo haría un servidor y a su vez reenviarlas como lo haría un cliente.

En esta actividad se implementó un proxy que filtra el contenido web con dos funcionalidades principales. Por un lado se bloquea el tráfico hacia páginas no permitidas como un control parental, y por otro se reemplaza contenido inadecuado sustituyendo ciertas palabras por otras. Todo se construyó exclusivamente con las librerías `socket`, `json` y `sys` de Python tal como lo exige el enunciado.

El proxy corre en una máquina virtual anclado a su IP, escuchando en el puerto 8000. Desde la máquina local se accede a él configurándolo como proxy del sistema o del navegador, o directamente con la herramienta `curl` usando la opción `-x`. A lo largo de este informe se documentan el funcionamiento y las decisiones de diseño de cada parte del proxy, siguiendo el orden de los pasos de la Parte 2 del enunciado.

Funciones base

La base del proxy son dos funciones que permiten convertir un mensaje HTTP en una estructura de datos y viceversa. La primera es `parse_HTTP_message`, que recibe el mensaje como bytes y lo transforma en un diccionario de Python para acceder fácilmente a la información. La segunda es `create_HTTP_message`, que hace el proceso inverso, toma el diccionario y reconstruye el mensaje HTTP en bytes.

La función `parse_HTTP_message` extrae la línea de inicio, los headers y el body. La línea de inicio contiene el método, el path o target y la versión del protocolo, o en el caso de una respuesta contiene la versión, el código de estado y la razón. Los headers quedan guardados en un diccionario donde cada nombre apunta a su valor. El body se conserva siempre como bytes sin decodificar, ya que puede contener información binaria que no debe corromperse al convertirse a texto.

```python
def parse_HTTP_message(http_message: bytes) -> dict:
    if b"\r\n\r\n" in http_message:
        head, body = http_message.split(b"\r\n\r\n", 1)
    else:
        head, body = http_message, b""
```

```
    lines = head.split(b"\r\n")
    start_line = lines[0].decode()
    parts = start_line.split()

    msg = {"headers": {}, "body": body}

    if parts[0].startswith("HTTP"):
        msg["is_request"] = False
        msg["version"] = parts[0]
        msg["status"] = parts[1]
        msg["reason"] = start_line.split(None, 2)[2] if len(parts) > 2 else ""
    else:
        msg["is_request"] = True
        msg["method"] = parts[0]
        msg["target"] = parts[1]
        msg["version"] = parts[2]

    for line in lines[1:]:
        name, value = line.decode().split(":", 1)
        msg["headers"][name.strip()] = value.strip()

    return msg
```

La función `create_HTTP_message` construye la línea de inicio según se trate de una petición o una respuesta, agrega cada header con el formato nombre espacio dos puntos espacio valor y separa cada línea con el salto de línea `\r\n`. Al final concatena el doble salto de línea `\r\n\r\n` que marca el fin de los headers y agrega el body.

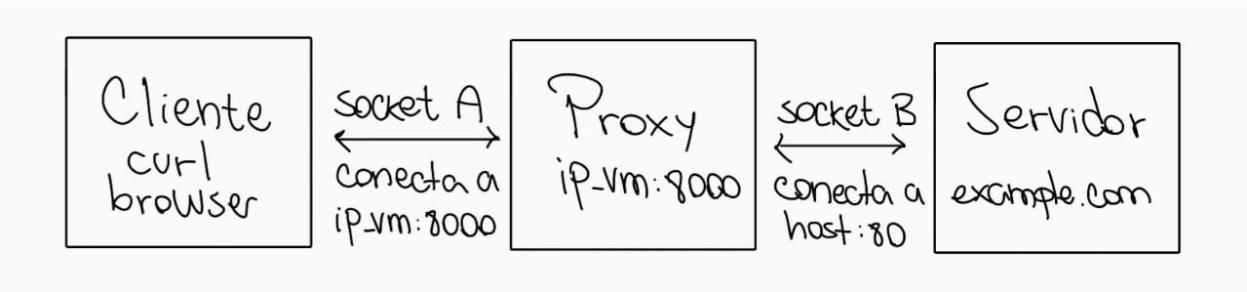
```
def create_HTTP_message(msg: dict) -> bytes:
    if msg["is_request"]:
        start_line = f"{msg['method']} {msg['target']} {msg['version']}"
    else:
        start_line = f"{msg['version']} {msg['status']} {msg['reason']}"
    lines = [start_line]
    for name, value in msg["headers"].items():
        lines.append(f"{name}: {value}")

    return "\r\n".join(lines).encode() + b"\r\n\r\n" + msg["body"]
```

Como decisión de diseño, los headers se guardan como diccionario, lo que facilita el acceso por nombre y la modificación posterior. Una limitación de esta elección es que si un header se repite, el último valor sobrescribe al anterior. El body se mantiene en bytes, por lo que el mensaje reconstruido es idéntico al original byte a byte.

Diagrama del proxy

Antes de implementar el proxy se dibujó el siguiente diagrama del flujo.



El proxy necesita dos sockets por conexión para comunicarse con un cliente que intenta acceder a un servidor. El primer socket es el del proxy actuando como servidor, con el que acepta al cliente mediante `accept` en el puerto 8000. Por este socket entra la petición del cliente y sale la respuesta hacia él. El segundo socket es el del proxy actuando como cliente, con el que se conecta mediante `connect` al servidor real en el puerto 80 indicado por la petición. Por este segundo socket sale la petición hacia el servidor y entra la respuesta desde él.

De esta forma el proxy puede recibir consultas como lo haría un servidor y reenviarlas como lo haría un cliente, manteniéndose siempre en el medio de la comunicación. La petición del cliente entra por el primer socket y sale por el segundo, y la respuesta del servidor entra por el segundo y sale por el primero.

Paso 1. Proxy transparente

El primer paso es convertir el servidor en un proxy que se interpone entre el cliente y el servidor real sin modificar el mensaje. El proxy recibe la petición, la envía al servidor de forma íntegra, recibe la respuesta y se la envía al cliente sin modificarla.

Para saber a qué servidor quiere llegar el cliente, el proxy obtiene el destino desde la petición. Cuando se usa un proxy, el navegador o `curl` escriben la URI absoluta en la línea de pedido, como `GET http://host/path HTTP/1.1`. Además siempre envían el header `Host`. La función `get_destination` revisa primero la URI absoluta del target y, si no la tiene, usa el header `Host`. Si la dirección no incluye un puerto, se asume el puerto 80 de HTTP.

```
def get_destination(parsed: dict) -> tuple:
    target = parsed["target"]
    if target.startswith("http://"):
        rest = target[len("http://"):]
        authority, _, remainder = rest.partition("/")
        path = "/" + remainder
    else:
```

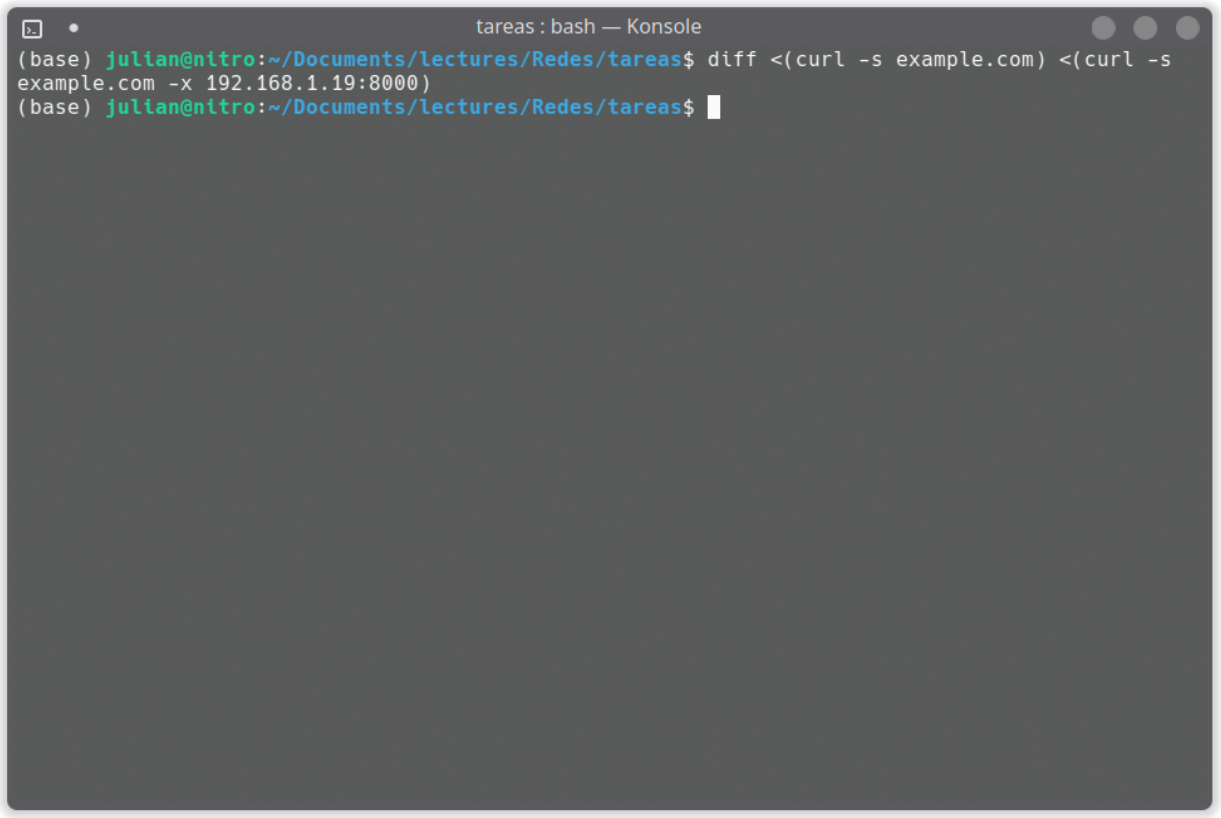
```
        authority = parsed["headers"].get("Host", "")
        path = target

    if ":" in authority:
        host, port = authority.rsplit(":", 1)
        return host, int(port), path

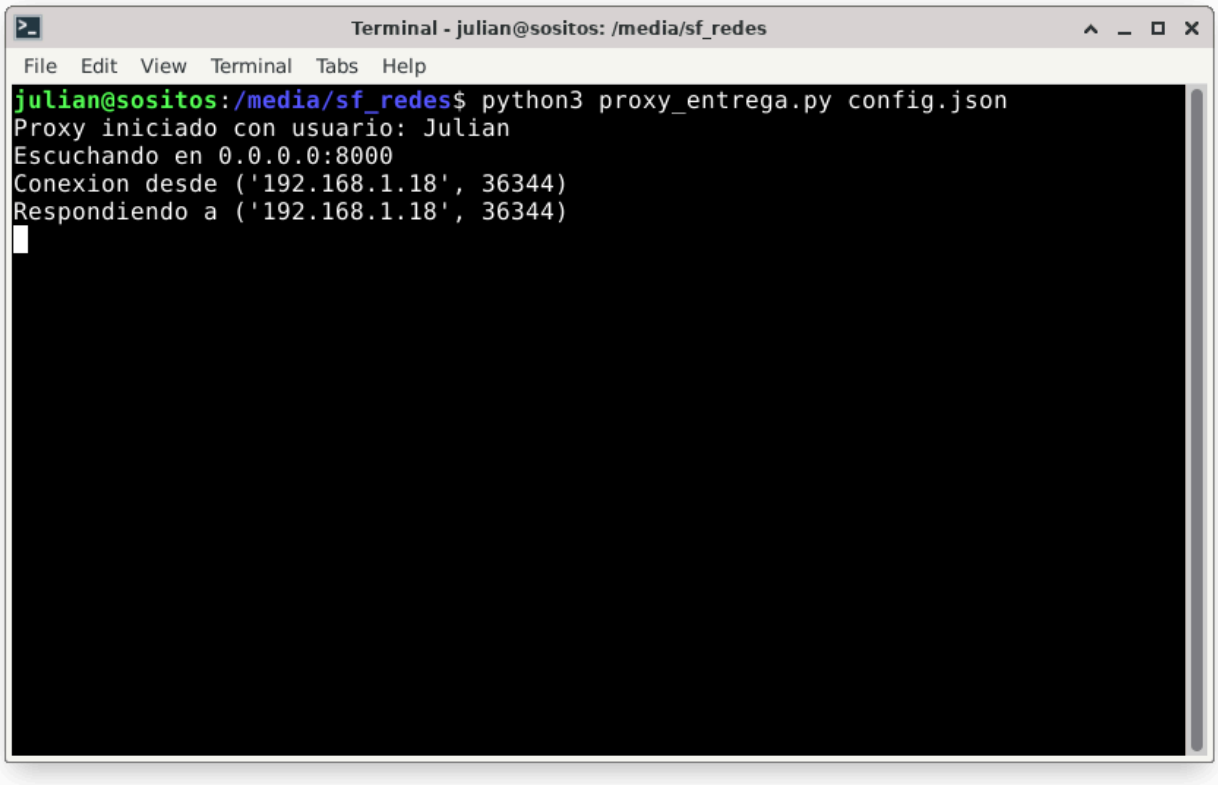
    return authority, 80, path
```

Con el destino ya conocido, el proxy crea un segundo socket, se conecta al servidor mediante `connect`, reenvía la petición con `sendall` y lee la respuesta con `recv_full_message`. Luego cierra el socket del servidor y envía la respuesta al cliente.

Para comprobar que el proxy transfiere los mensajes de forma intacta, se pidió la página `example.com` con `curl` sin proxy y con el proxy. Ambos comandos entregan el mismo contenido, por lo que la comparación no muestra ninguna diferencia.



La siguiente captura muestra el proxy corriendo en la máquina virtual, donde se ve cada conexión entrante, el reenvío del destino y la respuesta al cliente.



Paso 2. Bloqueo de dominios prohibidos

La segunda funcionalidad del proxy es bloquear el tráfico hacia páginas no permitidas, como un control parental. Para esto se usa la lista `blocked` del archivo de configuración JSON. La función `is_blocked` recibe el host y el path de la petición y los compara con cada entrada de la lista. Cada entrada puede ser un dominio exacto como `www.dcc.uchile.cl` o un dominio con ruta como `cc4303.bachmann.cl/secret`. Para el segundo caso se compara con la unión del host más el path.

```
def is_blocked(host: str, path: str, config: dict) -> bool:
    for entry in config.get("blocked", []):
        if entry == host or entry == host + path:
            return True
    return False
```

Si el destino está bloqueado, el proxy no reenvía la petición al servidor, sino que responde con el código de error 403 junto a un HTML que muestra una imagen guardada localmente en la máquina virtual. Para la imagen se usa la etiqueta `img` con el path `/gato.png`.

```
def create_403_response() -> dict:
    html = (
        "<!DOCTYPE html>\n"
```

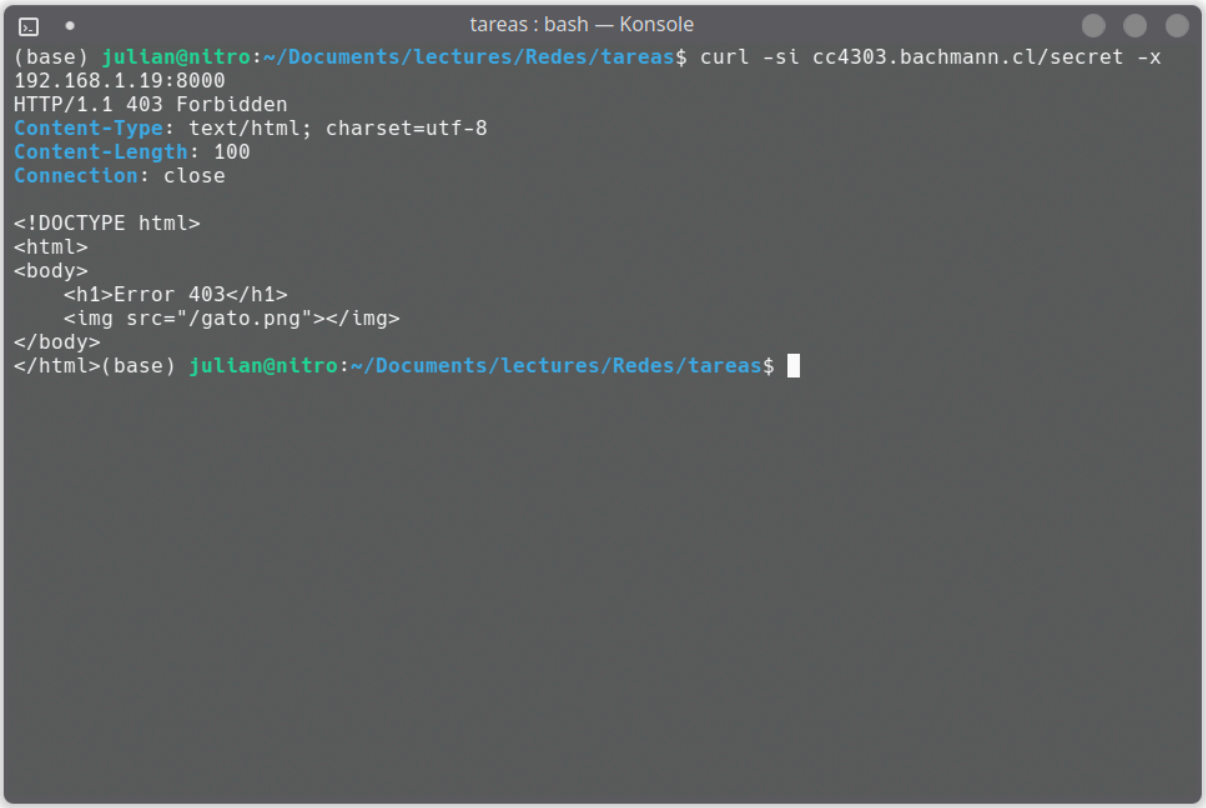
```
        "<html>\n"
        "<body>\n"
        "    <h1>Error 403</h1>\n"
        f'    <img src="{IMG_PATH}"></img>\n'
        "</body>\n"
        "</html>"
    )
    body = html.encode()
    return {
        "is_request": False,
        "version": "HTTP/1.1",
        "status": "403",
        "reason": "Forbidden",
        "headers": {
            "Content-Type": "text/html; charset=utf-8",
            "Content-Length": str(len(body)),
            "Connection": "close",
        },
        "body": body,
    }
```

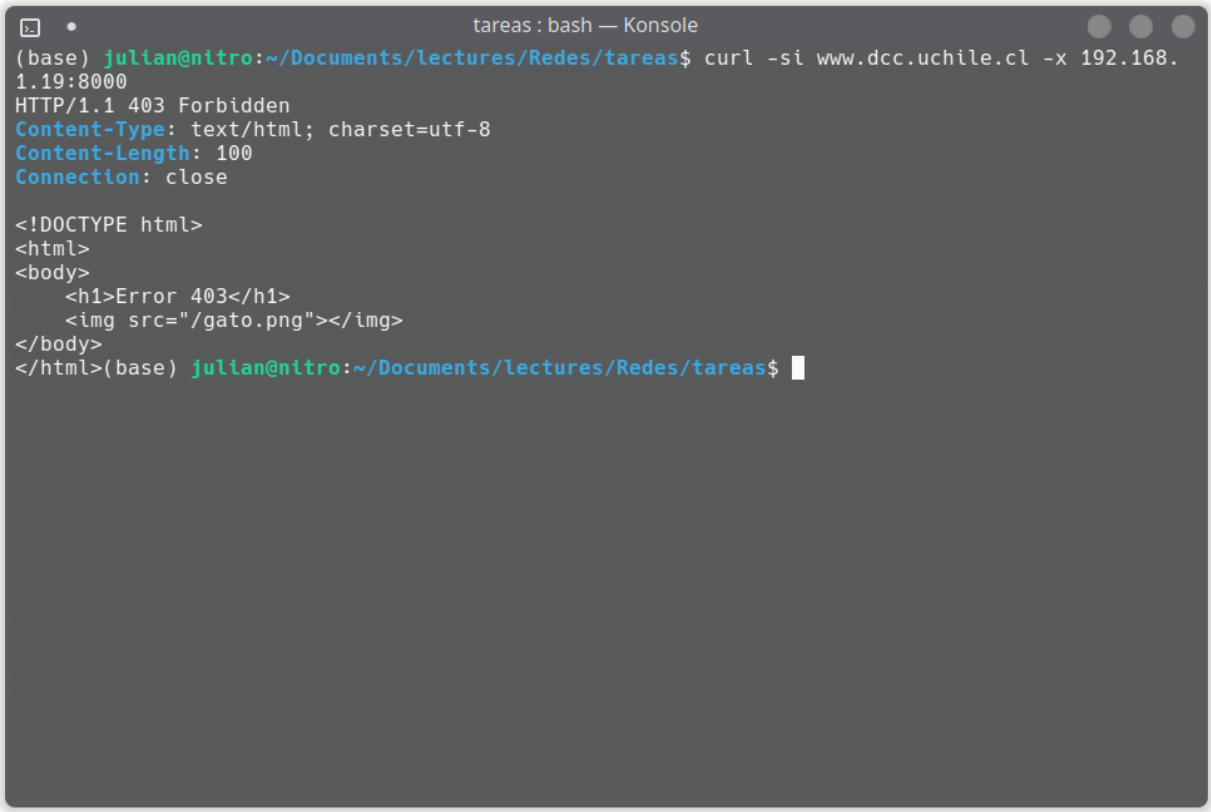
Para mostrar una imagen en un navegador son necesarios dos ciclos de comunicación HTTP. En el primero el navegador pide la página bloqueada y recibe el HTML del error 403, que contiene la etiqueta `img`. En el segundo ciclo, al interpretar el HTML, el navegador hace una nueva petición por la imagen `/gato.png` y el proxy la responde con el contenido de la imagen. Por eso el proxy atiende la petición de la imagen antes de revisar el bloqueo, evitando que la imagen quede bloqueada y provocando un ciclo interminable.

En la rama del código, si el path pedido coincide con el de la imagen se responde con el gato. Si no, se revisa si el dominio está bloqueado y se responde con el 403. Solo si no está bloqueado se procede a reenviar la petición.

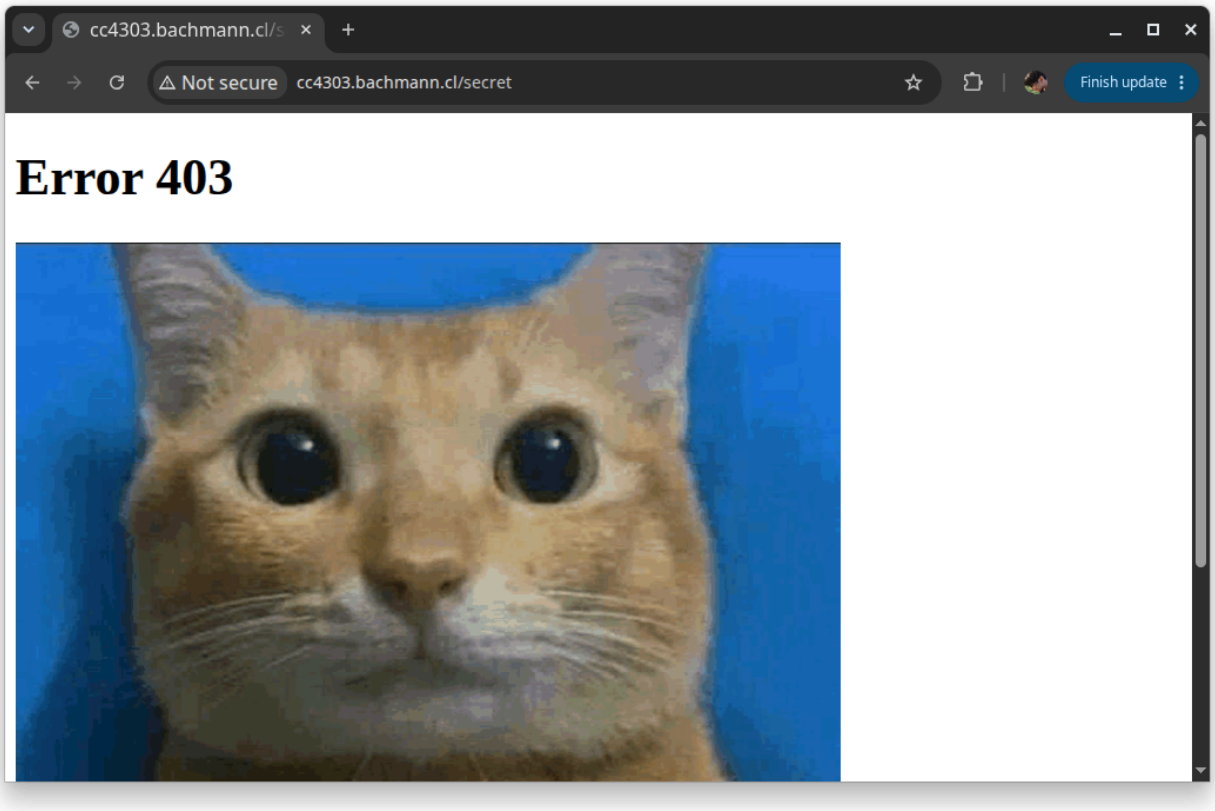
```
if path == IMG_PATH:
    response = create_HTTP_message(create_image_response())
elif is_blocked(host, path, config):
    response = create_HTTP_message(create_403_response())
```

La siguiente captura muestra cómo `curl` recibe el 403 al pedir la página prohibida `cc4303.bachmann.cl/secret` a través del proxy. También se probó con el dominio exacto `www.dcc.uchile.cl`, que está en la lista bloqueada.


tareas : bash — Konsole
(base) julian@nitro:~/Documents/lectures/Redes/tareas$ curl -si cc4303.bachmann.cl/secret -x 192.168.1.19:8000
HTTP/1.1 403 Forbidden
Content-Type: text/html; charset=utf-8
Content-Length: 100
Connection: close
<!DOCTYPE html>
<html>
<body>
<h1>Error 403</h1>
<img src="/gato.png"></img>
</body>
</html>(base) julian@nitro:~/Documents/lectures/Redes/tareas$


tareas : bash — Konsole
(base) julian@nitro:~/Documents/lectures/Redes/tareas$ curl -si www.dcc.uchile.cl -x 192.168.1.19:8000
HTTP/1.1 403 Forbidden
Content-Type: text/html; charset=utf-8
Content-Length: 100
Connection: close
<!DOCTYPE html>
<html>
<body>
<h1>Error 403</h1>
<img src="/gato.png"></img>
</body>
</html>(base) julian@nitro:~/Documents/lectures/Redes/tareas$

Al usar el proxy en el navegador, la página prohibida `cc4303.bachmann.cl/secret` ya no se muestra, sino que se recibe el error 403 y se ve la imagen del gato guardada localmente.



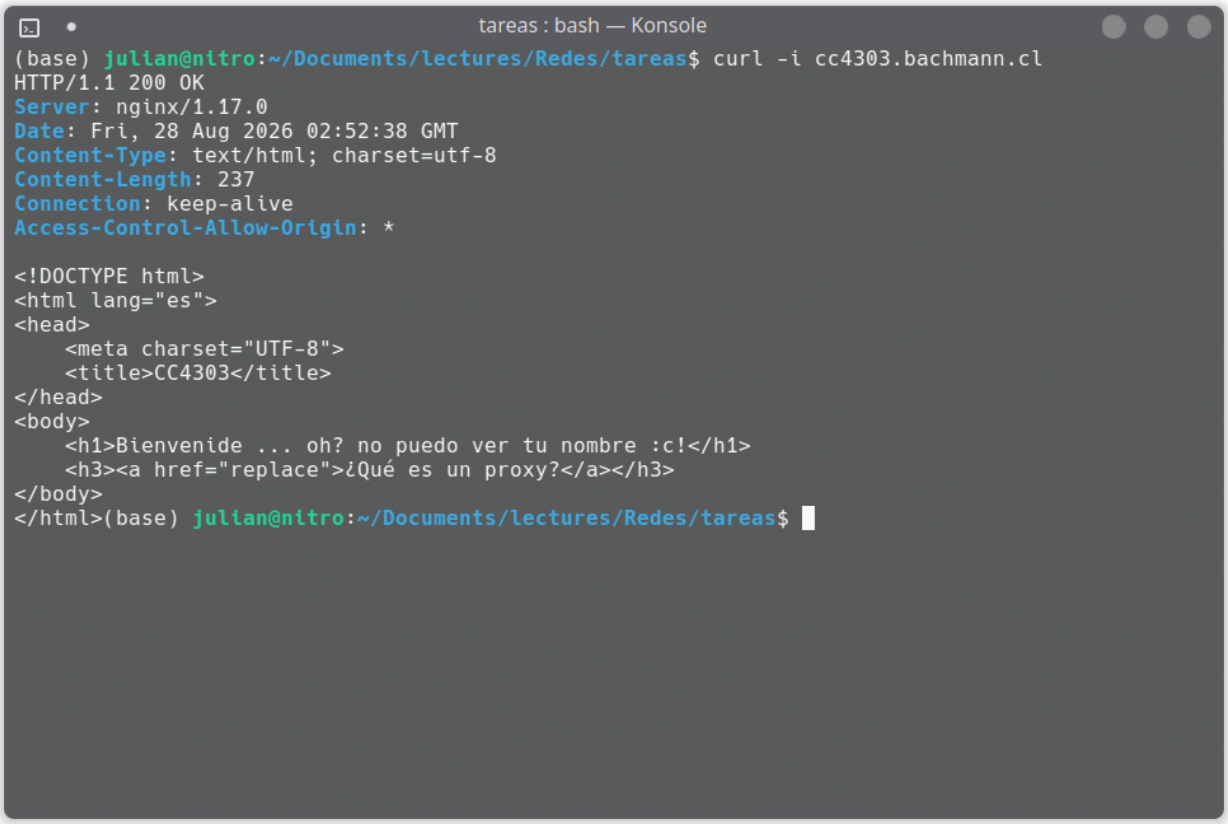
Paso 3. Header X-ElQuePregunta

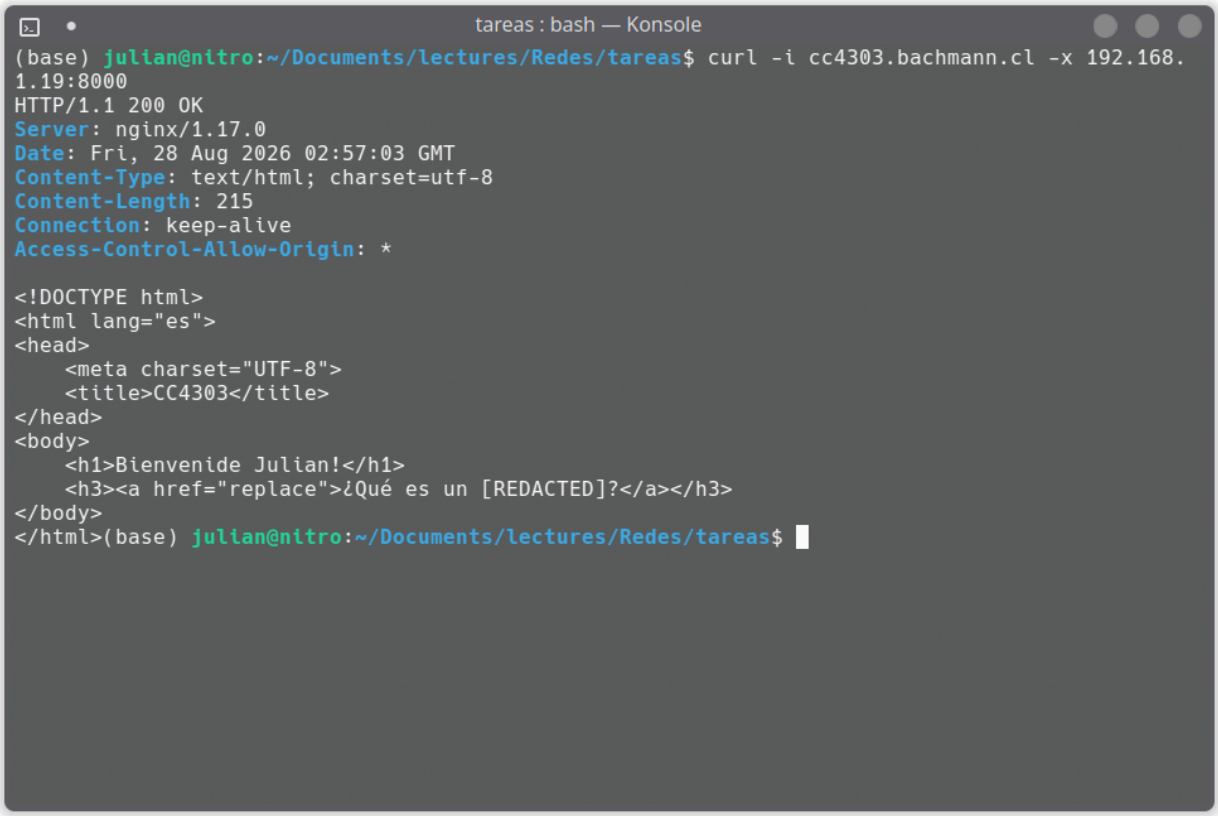
Cuando la dirección del servidor final no es una dirección prohibida, el proxy agrega a la petición que reenvía al servidor el header `X-ElQuePregunta` con el nombre del usuario. El nombre se obtiene del archivo de configuración JSON y se carga en la variable `user` al iniciar el proxy.

```
parsed["headers"]["X-ElQuePregunta"] = user
```

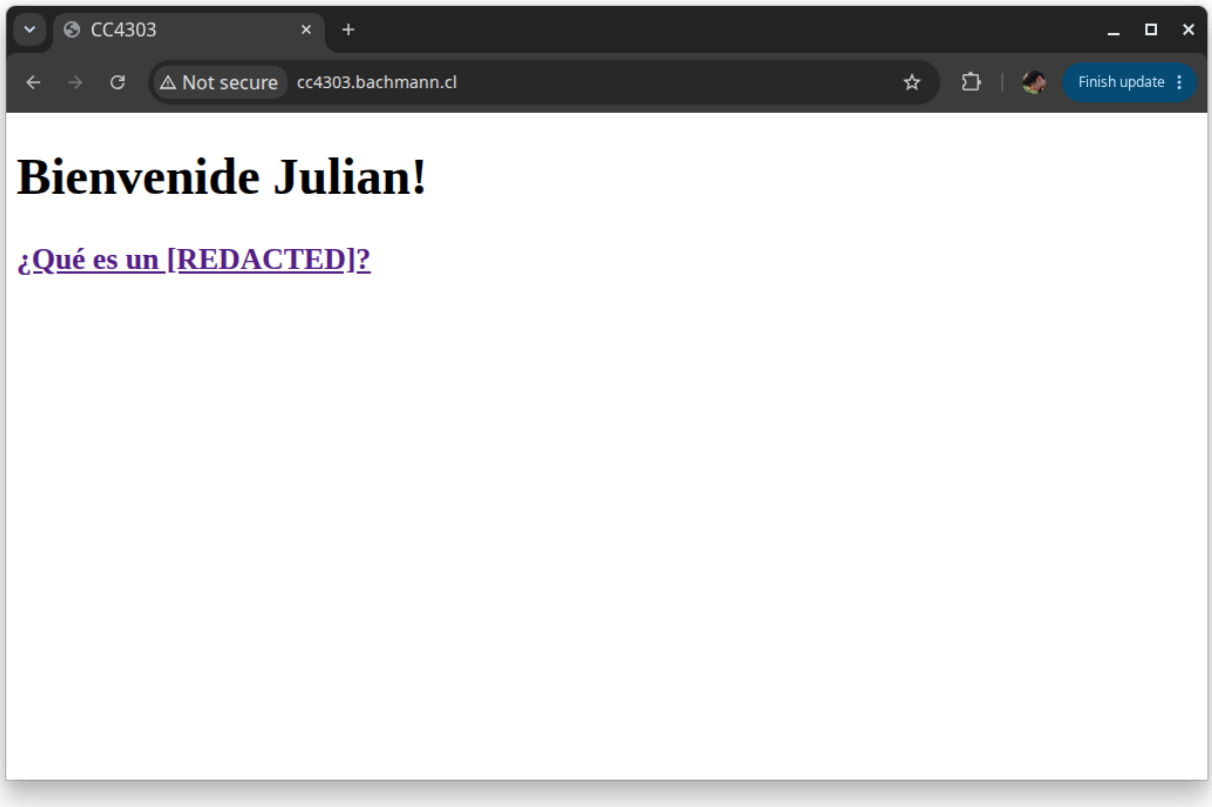
Luego se reconstruye la petición con `create_HTTP_message` y se reenvía al servidor. Como esta línea se encuentra dentro de la rama `else`, el header solo se agrega cuando el dominio no está bloqueado, cumpliendo con lo pedido por el enunciado.

El nombre del header debe ser exactamente `X-ElQuePregunta`, ya que la página de prueba lo detecta para mostrar el mensaje de bienvenida. La siguiente captura muestra la petición con `curl` sin pasar por el proxy, donde el mensaje de bienvenida no muestra el nombre. Al pedir la misma página a través del proxy, el header se agrega a la petición y el mensaje de bienvenida cambia mostrando el nombre del usuario.


tareas : bash — Konsole
(base) julian@nitro:~/Documents/lectures/Redes/tareas$ curl -i cc4303.bachmann.cl
HTTP/1.1 200 OK
Server: nginx/1.17.0
Date: Fri, 28 Aug 2026 02:52:38 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 237
Connection: keep-alive
Access-Control-Allow-Origin: *
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>CC4303</title>
</head>
<body>
<h1>Bienvenide ... oh? no puedo ver tu nombre :c!</h1>
<h3>¿Qué es un proxy?</h3>
</body>
</html>(base) julian@nitro:~/Documents/lectures/Redes/tareas$


tareas : bash — Konsole
(base) julian@nitro:~/Documents/lectures/Redes/tareas$ curl -i cc4303.bachmann.cl -x 192.168.1.19:8000
HTTP/1.1 200 OK
Server: nginx/1.17.0
Date: Fri, 28 Aug 2026 02:57:03 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 215
Connection: keep-alive
Access-Control-Allow-Origin: *
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>CC4303</title>
</head>
<body>
<h1>Bienvenide Julian!</h1>
<h3>¿Qué es un [REDACTED]?</h3>
</body>
</html>(base) julian@nitro:~/Documents/lectures/Redes/tareas$

En el navegador también se pudo ver el saludo al acceder a `cc4303.bachmann.cl` con el proxy configurado.



Paso 4. Reemplazo de palabras prohibidas

La cuarta funcionalidad del proxy es reemplazar contenido inadecuado. Para esto se usa la lista `forbidden_words` del archivo de configuración JSON, que contiene pares de palabra original y palabra de reemplazo. La función `apply_replacements` recorre la respuesta del servidor y sustituye cada palabra prohibida por su reemplazo.

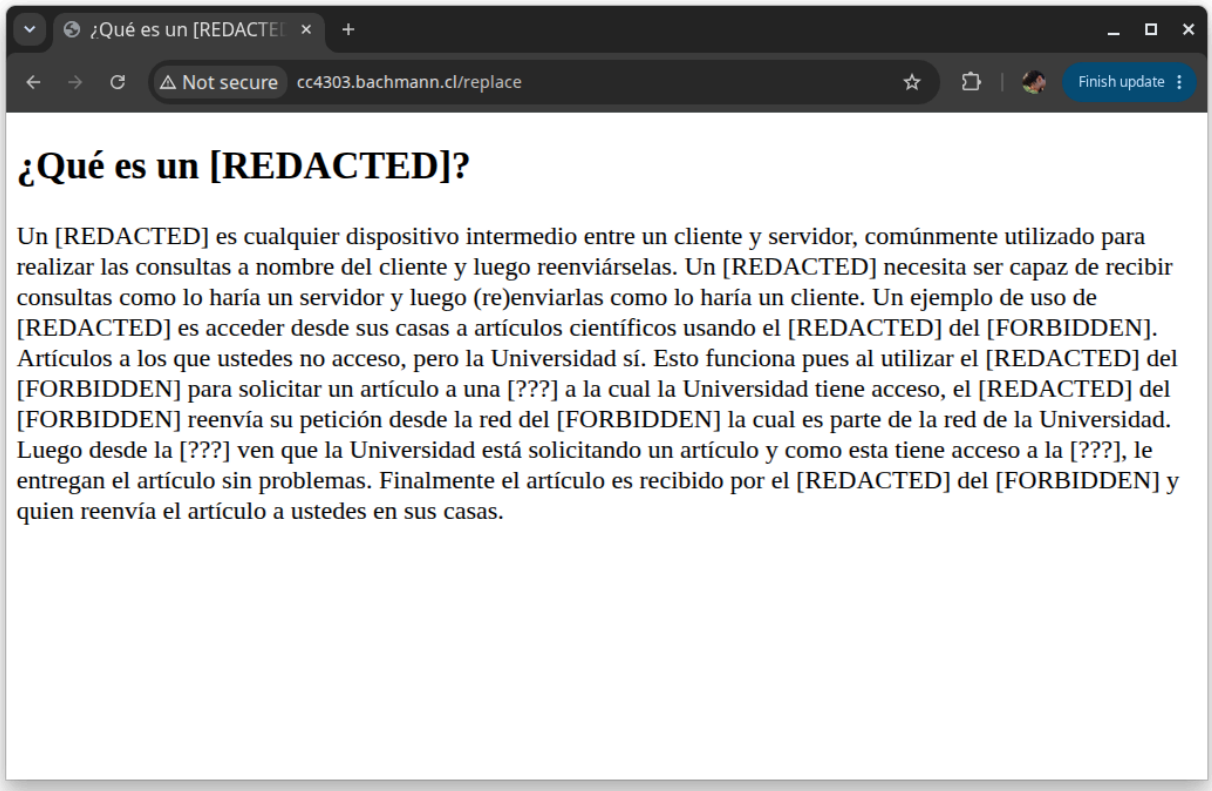
```
def apply_replacements(parsed_resp: dict, config: dict) -> dict:
    content_type = parsed_resp["headers"].get("Content-Type", "")
    if "text" not in content_type:
        return parsed_resp
    text = parsed_resp["body"].decode("utf-8", errors="replace")
    for pair in config.get("forbidden_words", []):
        for word, replacement in pair.items():
            text = text.replace(word, replacement)
    new_body = text.encode("utf-8")
    parsed_resp["body"] = new_body
    parsed_resp["headers"]["Content-Length"] = str(len(new_body))
    return parsed_resp
```

Hay dos decisiones de diseño importantes en esta función. La primera es que solo se reemplazan las palabras si el header `Content-Type` de la respuesta contiene la palabra `text`, lo que indica que el contenido es texto. Esto evita corromper imágenes u otros archivos binarios que pasen por el proxy. La segunda es que al modificar el body se debe recalcular el header `Content-Length`, ya que el largo del contenido cambia. Si no se actualizara, el cliente leería una cantidad de bytes distinta a la del body y la página quedaría cortada.

La siguiente captura muestra el resultado de pedir la página `cc4303.bachmann.cl/replace` a través del proxy, donde las palabras prohibidas aparecen reemplazadas por su versión censurada.



En el navegador también se verificó que el texto de la página es el mismo, solo que con las palabras prohibidas censuradas.



Paso 5. Manejo de mensajes más grandes que el buffer

Hasta ahora el proxy recibía los mensajes con un solo `recv`, lo que funciona cuando el mensaje cabe completo en el buffer del socket. Sin embargo, un mensaje HTTP puede ser mucho más grande que el buffer, por lo que un solo `recv` lo cortaría. Para resolverlo se implementó la función `recv_full_message`, que lee en un ciclo hasta que el mensaje está completo.

La estrategia se divide en tres fases. En la primera se acumulan trozos en un ciclo hasta que aparezca la secuencia `\r\n\r\n`, que marca el fin de los headers. En la segunda se busca el header `Content-Length` para conocer el largo exacto del body. En la tercera se sigue leyendo hasta completar el body según ese largo. Si no hay `Content-Length`, como en una respuesta HTTP antigua, se lee hasta que el servidor cierre la conexión o se agote un tiempo de espera de cinco segundos.

```
def recv_full_message(sock, buffer_size=4096):
    sock.settimeout(5)
    data = b""

    # fase 1: acumular trozos hasta que los headers esten completos
    while b"\r\n\r\n" not in data:
```

```
        try:
            chunk = sock.recv(buffer_size)
        except socket.timeout:
            return data
        if not chunk:
            return data
        data += chunk

    head, _, body = data.partition(b"\r\n\r\n")
    is_request = not head.startswith(b"HTTP")

    # fase 2: buscar Content-Length
    content_length = None
    for line in head.split(b"\r\n")[1:]:
        name, _, value = line.partition(b":")
        if name.strip().lower() == b"content-length":
            content_length = int(value.strip())

    # fase 3a: con Content-Length, leer exactamente ese resto del body
    if content_length is not None:
        while len(body) < content_length:
            chunk = sock.recv(buffer_size)
            if not chunk:
                break
            data += chunk
            body = data.partition(b"\r\n\r\n")[2]
        return data

    # fase 3b: request sin Content-Length (GET/HEAD) ya esta completa
    if is_request:
        return data

    # fase 3c: response sin Content-Length, leer hasta cerrar o timeout
    while True:
        try:
            chunk = sock.recv(buffer_size)
        except socket.timeout:
            break
        if not chunk:
            break
        data += chunk
    return data
```

Este diseño responde directamente a las cuatro preguntas planteadas por el enunciado.

Pregunta	**Respuesta**

¿Cómo sé si llegó el mensaje completo?	Los headers están completos cuando aparece la secuencia `\r\n\r\n`. El body está completo cuando se han leído exactamente los bytes indicados por `Content-Length`.
¿Qué pasa si los headers no caben en mi buffer?	El primer ciclo acumula varios trozos con la suma `data += chunk` hasta que aparece `\r\n\r\n`, sin importar cuántos `recv` se necesiten.
¿Cómo sé que el HEAD llegó completo?	Cuando la secuencia `\r\n\r\n` aparece en los datos acumulados, ese es justamente el delimitador que marca el fin de los headers.
¿Y el BODY?	Si existe `Content-Length`, se sigue leyendo hasta que el largo del body acumulado sea igual a ese valor. Si no existe, se lee hasta que el servidor cierre la conexión o se agote el tiempo de espera.

Para probar que el proxy sigue funcionando con un buffer pequeño, se cambió el tamaño del buffer a 50 bytes y se pidió la página `cc4303.bachmann.cl/replace`, cuyo contenido es de 1155 bytes. Con un buffer de 50 se necesitan alrededor de 24 lecturas para recibir el mensaje completo, y el reemplazo de palabras sigue funcionando correctamente.



También se probaron los dos casos mencionados por el enunciado. En el primero, el buffer es más pequeño que el tamaño total del mensaje pero más grande que el área de headers, y el proxy funciona bien. En el segundo, el buffer es más pequeño que el área de headers pero más grande que la línea de inicio, y el proxy también funciona, pues la primera fase acumula varios trozos hasta completar los headers.

Pruebas con navegador

Para probar el proxy con un navegador se configuró la dirección `IP_VM:8000` como proxy del sistema. Con esto el navegador envía todas sus peticiones al proxy, que las procesa con las funcionalidades ya descritas.

Con el proxy configurado se verificó el bloqueo de la página `cc4303.bachmann.cl/secret`, que ya no se muestra, sino que se recibe el error 403 y se muestra la imagen del gato guardada localmente. Esta prueba se puede ver en el paso 2, junto con la captura correspondiente.

También se verificó que al acceder a `cc4303.bachmann.cl` el contenido se modifica según los cambios introducidos en los headers. En este caso se muestra el mensaje de bienvenida con el nombre del usuario, gracias al header `X-ElQuePregunta` agregado por el proxy. La captura se encuentra en el paso 3.

Finalmente se verificó que los sitios `cc4303.bachmann.cl` y `cc4303.bachmann.cl/replace` se pueden ver, pero las palabras prohibidas aparecen censuradas según el archivo de configuración. El texto es el mismo, solo que con las palabras prohibidas reemplazadas, tal como se muestra en la captura del paso 4.

Luego de terminar las pruebas se quitó la configuración del proxy del navegador para no afectar la navegación normal.

Cómo ejecutar el código

El proxy se ejecuta en la máquina virtual desde la carpeta donde se encuentran el código y la imagen. Para iniciarlo se usa el siguiente comando, indicando el archivo de configuración como argumento.

```
python3 proxy_entrega.py config.json
```

El proxy queda escuchando en el puerto 8000 de todas las interfaces de la máquina virtual, por lo que es alcanzable desde la máquina local mediante la IP de la máquina virtual.

Para probar el proxy con `curl` se usa la opción `-x` indicando la IP de la máquina virtual y el puerto.

```
curl cc4303.bachmann.cl -x IP_VM:8000
```

Para probar el proxy con el navegador se configura la dirección `IP_VM:8000` como proxy del sistema o del navegador. El archivo `config.json` contiene el usuario, la lista de dominios bloqueados y las palabras prohibidas con sus reemplazos.

Repositorio de GitHub

El código de la actividad está disponible en el repositorio de GitHub al que el equipo docente tiene acceso. El enlace al repositorio es el siguiente.

[https://github.com/julianticucho/cc4303_tareas]

Y la rama es `control1parte1`

Declaración de uso de IA

Se utilizó Gemini de Google como apoyo para documentar el uso de Python, principalmente para consultar cómo trabajar con strings, bytes y manejo de sockets en Python. También se uso el autocompletado que viene por defecto en VSCode. El código, las decisiones de diseño y las pruebas fueron escritas y validadas de forma manual para asegurar su correcto funcionamiento.